Keyword Spotting on Raspberry Pi 3

A Technical Report on Decoder-Only Transformer Training
and Post-Training Quantization via AWQ and SmoothQuant

Zentree Labs

May 1, 2026

Contents

1 Introduction **2**

2 Dataset **2**
2.1 Google Speech Commands v2 . . . 2
2.2 Dataset Index Construction . . . 2

3 Audio Preprocessing **3**
3.1 Resampling . . . 3
3.2 Duration Normalization . . . 3
3.3 Short-Time Fourier Transform . . . 3
3.4 Mel Filterbank . . . 4
3.5 Log Compression . . . 4
3.6 Per-Sample Normalization . . . 4
3.7 Data Augmentation . . . 5

4 Model Architecture **5**
4.1 Overview . . . 5
4.2 Input Projection . . . 5
4.3 CLS Token and Positional Embedding . . . 5
4.4 Causal Self-Attention . . . 6
4.5 Feed-Forward Network . . . 7
4.6 Decoder Block . . . 7
4.7 Classification Head . . . 7
4.8 Weight Initialization . . . 7
4.9 Parameter Count Breakdown . . . 8

5 Training **8**
5.1 Loss Function . . . 8
5.2 Optimizer . . . 8
5.3 Learning Rate Schedule . . . 9
5.4 Gradient Clipping . . . 9
5.5 Training Procedure . . . 9

6 Post-Training Quantization **10**
6.1 Motivation . . . 10
6.2 Quantization Fundamentals . . . 10
6.2.1 reduce_range for ARM NEON . . . 10
6.3 SmoothQuant . . . 11
6.3.1 Motivation . . . 11
6.3.2 Key Idea . . . 11
6.3.3 Scale Computation . . . 11
6.3.4 Activation Calibration . . . 12
6.3.5 Applied Layers . . . 12
6.4 AWQ: Activation-Aware Weight Quantization . . . 12
6.4.1 Motivation . . . 12
6.4.2 Pseudo-Quantization . . . 13
6.4.3 Grid Search Objective . . . 13
6.4.4 Scale Application . . . 13
6.4.5 Relationship Between AWQ and SmoothQuant . . . 14
6.5 ONNX Export . . . 14
6.6 ONNX Runtime Static INT8 Quantization . . . 14
6.6.1 Overview . . . 14
6.6.2 Calibration . . . 14
6.6.3 Quantized Operator Selection . . . 15
6.6.4 Quantization Settings . . . 15
6.6.5 QDQ Graph Structure . . . 16

7 Deployment on Raspberry Pi 3 **16**
7.1 Hardware Constraints . . . 16
7.2 Runtime Configuration . . . 16
7.3 Real-Time Inference . . . 16
7.4 Preprocessing on the Pi . . . 17

8 Summary **17**

1 Introduction

This document provides a complete technical description of a keyword spotting system designed for deployment on a Raspberry Pi 3. The system recognizes 35 spoken keywords from the Google Speech Commands v2 dataset using a decoder-only transformer model that has been post-training quantized to 8-bit integers (INT8) for efficient inference on ARM hardware.

The report is organized as follows. Section 2 describes the dataset. Section 3 details every step of the audio preprocessing pipeline. Section 4 presents the model architecture in full, including the motivation for each design choice. Section 5 documents the training procedure. Section 6 covers post-training quantization, explaining SmoothQuant, AWQ, ONNX export, and ONNX Runtime static INT8 quantization in depth. Section 7 addresses deployment considerations for the Raspberry Pi 3.

2 Dataset

2.1 Google Speech Commands v2

The model is trained on **Google Speech Commands v2** (`speech_commands_v0.02`), a publicly available dataset of one-second audio clips containing single spoken English words. The dataset was introduced by Warden (2018) and contains approximately 105,000 utterances across 35 word classes, recorded at 16 kHz by thousands of speakers.

Table 1: Dataset statistics for the 35-keyword subset used in this work.

Split	Utterances	Approx. duration
Training	84,843	≈23.6 h
Validation	9,981	≈2.8 h
Test	11,005	≈3.1 h

The 35 target keywords are:

`backward, bed, bird, cat, dog, down, eight, five, follow, forward, four, go, happy, house, learn, left, marvin, nine, no, off, on, one, right, seven, sheila, six, stop, three, tree, two, up, visual, wow, yes, zero`

The dataset is split into training, validation, and test sets using the official speaker-disjoint split files included with the download. Clips that do not belong to the 35 keywords (background noise samples and other utterances) are discarded during index construction.

2.2 Dataset Index Construction

A key implementation detail is how the dataset index is built. A naive approach iterates over every sample and decodes its audio to read the label, which is prohibitively slow (∼105,000 torchcodec decode calls). Instead, the label is read directly from the file-system path — the parent directory name of each `.wav` file equals the spoken word — without loading any audio:

```
self._indices = [
    i for i, path in enumerate(self._ds._walker)
    if Path(path).parent.name in LABEL2IDX
]
```

Listing 1: Fast index construction using file paths.

This reduces index construction time from several minutes to under one second.

3 Audio Preprocessing

Each raw waveform is converted into a log-mel spectrogram before being fed to the transformer. The preprocessing pipeline is identical at training time (PyTorch / `torchaudio`) and at inference time on the Raspberry Pi (NumPy / `librosa`), which is critical for avoiding a train–test preprocessing mismatch.

3.1 Resampling

All audio is resampled to $f_s = 16{,}000\,\text{Hz}$. Files already at 16 kHz incur no resampling cost. Resampling is performed with `torchaudio.functional.resample`, which uses a sinc interpolation filter by default.

3.2 Duration Normalization

All clips are normalized to exactly one second (16,000 samples):

- **Short clips** are zero-padded on the right: $\hat{x}[n] = x[n]$ for $n < L$, $\hat{x}[n] = 0$ otherwise.
- **Long clips** are truncated to the first 16,000 samples.

3.3 Short-Time Fourier Transform

The padded waveform $\hat{x} \in \mathbb{R}^{16000}$ is processed with a Short-Time Fourier Transform (STFT). The power spectrum at frame t and frequency bin k is:

$$P[t,k] = \left| \sum_{n=0}^{N-1} \hat{x}[n + t \cdot H] \cdot w[n] \cdot e^{-j2\pi kn/N} \right|^2 \tag{1}$$

where the STFT parameters are:

Table 2: STFT hyperparameters.

Parameter	Symbol	Value
FFT size	N	512
Window length	W	400 samples (25 ms)
Hop length	H	160 samples (10 ms)
Window function	w	Hann

The number of output frames for a one-second clip is:

$$T = \left\lfloor \frac{16000 - N}{H} \right\rfloor + 1 = 101 \text{ frames} \tag{2}$$

The number of frequency bins is $N/2 + 1 = 257$.

3.4 Mel Filterbank

The power spectrum is projected onto $M = 80$ mel-spaced triangular filters. The mel scale maps linear frequency f to a perceptually uniform scale:

$$m(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right) \tag{3}$$

The $M + 2$ mel-scale center frequencies are uniformly spaced between $m(0\,\text{Hz})$ and $m(8{,}000\,\text{Hz})$ (the Nyquist frequency at 16 kHz). These are converted back to linear frequencies and then to FFT bin indices. The i-th triangular filter is:

$$H_i[k] = \begin{cases} \dfrac{k - b_{i-1}}{b_i - b_{i-1}} & b_{i-1} \le k < b_i \\ \dfrac{b_{i+1} - k}{b_{i+1} - b_i} & b_i \le k < b_{i+1} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

where b_i is the FFT bin index of the i-th center frequency. The mel spectrogram is:

$$S[t, i] = \sum_k H_i[k] \cdot P[t, k] \quad \in \mathbb{R}^{101 \times 80} \tag{5}$$

3.5 Log Compression

Raw mel energies span several orders of magnitude. A logarithmic compression is applied with a small floor $\epsilon = 10^{-6}$ to prevent numerical issues at near-zero energies:

$$\tilde{S}[t, i] = \log(S[t, i] + \epsilon) \tag{6}$$

3.6 Per-Sample Normalization

Each sample is normalized to zero mean and unit variance independently (instance normalization):

$$\hat{S}[t, i] = \frac{\tilde{S}[t, i] - \mu}{\sigma + \delta} \qquad \mu = \frac{1}{T \cdot M} \sum_{t,i} \tilde{S}[t, i], \quad \sigma = \sqrt{\frac{1}{TM} \sum_{t,i} (\tilde{S}[t, i] - \mu)^2} \tag{7}$$

where $\delta = 10^{-5}$ prevents division by zero. Per-sample normalization is preferable to dataset-level statistics for keyword spotting because it naturally handles different recording conditions, microphone gains, and background noise levels at inference time.

The final spectrogram $\hat{S} \in \mathbb{R}^{101\times 80}$ is the input to the model. The time axis is treated as the sequence dimension and the mel axis as the feature dimension.

3.7 Data Augmentation

During training only, **SpecAugment** [1] is applied as a regularizer after normalization. Two masking operations are applied:

- **Frequency masking**: a contiguous block of up to $F = 10$ mel bins is zeroed out. The starting bin f_0 is drawn uniformly from $[0, M - F]$.
- **Time masking**: a contiguous block of up to $T' = 20$ frames is zeroed out. The starting frame t_0 is drawn uniformly from $[0, T - T']$.

Masking is applied in-place on the normalized spectrogram, which encourages the model to be robust to partial occlusion of both frequency and temporal information.

4 Model Architecture

4.1 Overview

The model is a **decoder-only transformer** adapted for classification. The key hyperparameters are summarized below:

Table 3: Model hyperparameters.

Hyperparameter	Symbol	Value
Model dimension	d	256
Number of heads	H	4
Head dimension	d_h	64
Number of layers	L	4
Feed-forward dim	d_{ff}	512
Dropout rate	p	0.1
Number of classes	C	35
Max sequence length	T_{max}	101
Total parameters		2,165,027

4.2 Input Projection

The mel spectrogram $\hat{S} \in \mathbb{R}^{T\times M}$ is first projected from the mel dimension $M = 80$ to the model dimension $d = 256$ by a learned linear layer:

$$\mathbf{X}_0 = \hat{S}\mathbf{W}_{in} + \mathbf{b}_{in} \quad \in \mathbb{R}^{T\times d} \qquad \mathbf{W}_{in} \in \mathbb{R}^{M\times d} \tag{8}$$

4.3 CLS Token and Positional Embedding

A learnable **classification token** $\mathbf{c} \in \mathbb{R}^{1\times d}$ is appended at the *end* of the sequence (not the beginning as in BERT):

$$\tilde{\mathbf{X}}_0 = \left[\mathbf{X}_0 \mid \mathbf{c}\right] \quad \in \mathbb{R}^{(T+1)\times d} \tag{9}$$

Why append at the end? In a causally-masked transformer, each position can only attend to earlier positions. If the CLS token is at position T (the last position), it attends to all T spectrogram frames and can therefore aggregate global information from the entire utterance. A CLS token at position 0 would only attend to itself and produce a degenerate representation.

Learned positional embeddings are added to distinguish frame positions:

$$\tilde{\mathbf{X}} = \tilde{\mathbf{X}}_0 + \mathbf{E}_{pos}[0 : T+1] \qquad \mathbf{E}_{pos} \in \mathbb{R}^{(T_{max}+1)\times d} \tag{10}$$

The positional embedding matrix is trained jointly with the rest of the model.

4.4 Causal Self-Attention

Each decoder block contains a **multi-head causal self-attention** layer. The input sequence $\mathbf{X} \in \mathbb{R}^{(T+1)\times d}$ is projected to queries, keys, and values via a single fused linear layer:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \text{split}(\mathbf{X}\mathbf{W}_{qkv} + \mathbf{b}_{qkv}) \qquad \mathbf{W}_{qkv} \in \mathbb{R}^{d\times 3d} \tag{11}$$

The fused projection avoids three separate matrix multiplies and is more cache-friendly. The output is split into three tensors each of shape $(T+1) \times d$ and then reshaped for multi-head processing.

For each attention head $h \in \{1, \ldots, H\}$, the sub-matrices $\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h \in \mathbb{R}^{(T+1)\times d_h}$ are extracted ($d_h = d/H = 64$). The scaled dot-product attention is:

$$\mathbf{A}_h = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_h}} + \mathbf{M}_{causal}\right)\mathbf{V}_h \quad \in \mathbb{R}^{(T+1)\times d_h} \tag{12}$$

where $\mathbf{M}_{causal} \in \mathbb{R}^{(T+1)\times(T+1)}$ is the causal mask:

$$[\mathbf{M}_{causal}]_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -10^4 & \text{if } i < j \end{cases} \tag{13}$$

The mask value -10^4 effectively zeros out future positions after softmax (since $e^{-10000} \approx 0$) while avoiding $-\infty$ in the computation graph, which would corrupt INT8 quantization calibration (see Section 6).

The outputs from all heads are concatenated and projected back to d:

$$\text{MHA}(\mathbf{X}) = [\mathbf{A}_1 \| \mathbf{A}_2 \| \cdots \| \mathbf{A}_H]\, \mathbf{W}_{out} + \mathbf{b}_{out} \qquad \mathbf{W}_{out} \in \mathbb{R}^{d\times d} \tag{14}$$

Attention dropout with probability $p = 0.1$ is applied to the attention weight matrix before multiplying by $\mathbf{V}_h$.

4.5 Feed-Forward Network

Each decoder block also contains a position-wise **feed-forward network** (FFN):

$$\mathrm{FFN}(\mathbf{x}) = \mathrm{Dropout}(\mathbf{W}_2 \cdot \mathrm{Dropout}(\mathrm{GELU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2) \tag{15}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{ff}}$ expands to $d_{ff} = 512$ and $\mathbf{W}_2 \in \mathbb{R}^{d_{ff} \times d}$ projects back. The expansion ratio is $d_{ff}/d = 2$.

GELU activation. The Gaussian Error Linear Unit (GELU) is used instead of ReLU:

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \approx 0.5x\Big(1 + \tanh\Big[\sqrt{2/\pi}\,\big(x + 0.044715x^3\big)\Big]\Big) \tag{16}$$

where Φ is the standard normal CDF. GELU produces smoother gradients than ReLU and has been found to perform better in transformer models.

4.6 Decoder Block

One decoder block applies pre-norm layer normalization, attention, residual connection, a second pre-norm, and the FFN:

$$\mathbf{X}' = \mathbf{X} + \mathrm{Dropout}(\mathrm{MHA}(\mathrm{LN}_1(\mathbf{X}))) \tag{17}$$

$$\mathbf{X}'' = \mathbf{X}' + \mathrm{FFN}(\mathrm{LN}_2(\mathbf{X}')) \tag{18}$$

Pre-norm (normalizing the input before the sub-layer) stabilizes training at large learning rates compared to post-norm (normalizing after the residual addition).

$L = 4$ identical decoder blocks are stacked sequentially.

4.7 Classification Head

After the final decoder block, a layer normalization is applied to the full sequence output. Only the CLS token (last position) is passed to the classification head:

$$\hat{\mathbf{y}} = \mathrm{LN}(\mathbf{X}^{(L)})_{[T,:]} \cdot \mathbf{W}_{cls} + \mathbf{b}_{cls} \quad \in \mathbb{R}^C \tag{19}$$

where $\mathbf{W}_{cls} \in \mathbb{R}^{d \times C}$ with $C = 35$.

4.8 Weight Initialization

All linear layers use truncated normal initialization with standard deviation $\sigma = 0.02$ (clamped to $\pm 2\sigma$), which is the GPT-2 initialization scheme. Biases are initialized to zero. LayerNorm weights are initialized to one and biases to zero. The CLS token is initialized to zero.

4.9 Parameter Count Breakdown

Table 4: Parameter breakdown by component.

Component	Formula	Parameters
Input projection	$M \cdot d + d$	20,736
Positional emb.	$(T_{max} + 1) \cdot d$	26,112
CLS token	d	256
Per decoder block ($\times 4$):		
LN_1	$2d$	512
QKV proj	$d \cdot 3d + 3d$	197,376
Out proj	$d^2 + d$	65,792
LN_2	$2d$	512
FFN up	$d \cdot d_{ff} + d_{ff}$	131,584
FFN down	$d_{ff} \cdot d + d$	131,328
Block total		527,104
4 blocks		2,108,416
Final LN	$2d$	512
Head	$d \cdot C + C$	8,995
Total		**2,165,027**

5 Training

5.1 Loss Function

The model is trained with **cross-entropy loss with label smoothing**:

$$\mathcal{L} = -\sum_{c=1}^{C} \tilde{y}_c \log p_c \qquad \tilde{y}_c = \begin{cases} 1 - \varepsilon + \varepsilon/C & \text{if } c = y \\ \varepsilon/C & \text{otherwise} \end{cases} \tag{20}$$

where y is the true label, $p_c = \text{softmax}(\hat{\mathbf{y}})_c$, and the smoothing factor is $\varepsilon = 0.1$. Label smoothing prevents the model from becoming overconfident and improves calibration of the output probabilities.

5.2 Optimizer

AdamW is used with the following hyperparameters:

Table 5: AdamW hyperparameters.

Hyperparameter	Value
Peak learning rate	3×10^{-4}
Weight decay	10^{-4}
β_1	0.9
β_2	0.98
ε	10^{-8} (PyTorch default)

AdamW decouples weight decay from the gradient update, preventing the interaction between adaptive learning rates and L2 regularization that occurs in the original Adam.

5.3 Learning Rate Schedule

The **OneCycleLR** scheduler is used, which linearly warms up the learning rate from $\sim$0 to the peak $\eta_{max} = 3 \times 10^{-4}$ over the first $\sim$30% of training, then anneals it to $\eta_{max}/10^4$ using a cosine schedule over the remaining 70%:

$$\eta(t) = \begin{cases} \dfrac{t}{t_{warmup}} \cdot \eta_{max} & \text{if } t \leq t_{warmup} \\ \eta_{min} + \dfrac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos\left(\pi \cdot \dfrac{t - t_{warmup}}{t_{total} - t_{warmup}}\right)\right) & \text{otherwise} \end{cases} \tag{21}$$

The schedule operates at the *step* level (updated every batch), not the epoch level.

5.4 Gradient Clipping

Gradient clipping is applied before each optimizer step to prevent exploding gradients:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{1}{\|\mathbf{g}\|_2}\right) \tag{22}$$

The maximum gradient norm is set to 1.0.

5.5 Training Procedure

Table 6: Training configuration.

Hyperparameter	Value
Batch size	256
Epochs	40
DataLoader workers	4
Device	CUDA (falls back to CPU)

At the end of each epoch, the model is evaluated on the validation set. If validation accuracy improves, the checkpoint is saved to `checkpoints/best_model.pt` with the following fields:

- `epoch` — epoch number
- `model_state` — state dict
- `val_acc` — validation accuracy at save time
- `config` — model hyperparameters (needed to reconstruct the model at quantization time)

Note that the dropout rate in the saved config is set to 0.0 (not the training rate of 0.1), because quantization and inference should use a deterministic model with no dropout.

6 Post-Training Quantization

6.1 Motivation

On the Raspberry Pi 3, the ARM Cortex-A53 processor has no hardware floating-point acceleration beyond the basic VFPv4 unit. INT8 arithmetic is significantly faster because:

1. The NEON SIMD unit can process 16 INT8 operations per cycle vs 4 FP32 operations — a 4× throughput advantage.
2. Model weights compress from ∼8.3 MB (FP32) to ∼2.1 MB (INT8), improving cache utilization.
3. DRAM bandwidth is reduced by 4×.

Post-training quantization (PTQ) is used rather than quantization-aware training (QAT) to avoid the cost of re-training.

6.2 Quantization Fundamentals

Symmetric per-channel INT8 quantization maps a floating-point tensor $\mathbf{w}$ to an integer tensor $\mathbf{w}_q$ as follows. For each output channel i:

$$s_i = \frac{\max_j |w_{ij}|}{q_{max}} \qquad q_{max} = 2^{b-1} - 1 = 127 \tag{23}$$

$$w_{ij}^q = \text{clamp}\left(\left\lfloor \frac{w_{ij}}{s_i} \right\rceil, -q_{max} - 1, q_{max} \right) \tag{24}$$

$$\hat{w}_{ij} = w_{ij}^q \cdot s_i \qquad \text{(dequantized)} \tag{25}$$

The quantization error is $\mathbf{w} - \hat{\mathbf{w}}$. For activations, the scale is determined from calibration data (see Section 6.6).

6.2.1 reduce_range for ARM NEON

On ARM Cortex-A53, the NEON dot-product instruction accumulates N INT8×INT8 products into a 32-bit accumulator. If both operands use the full INT8 range $[-128, 127]$,

the worst-case product is $(-128) \times (-128) = 16384$, and summing 256 such products gives 4,194,304, which can overflow a 16-bit intermediate register on some micro-architectures. The `reduce_range=True` flag shrinks the effective range to $[-64, 63]$ (7-bit), ensuring all intermediate products fit safely.

6.3 SmoothQuant

6.3.1 Motivation

The fundamental challenge of quantizing transformer activations is that different input channels can have vastly different magnitudes. Consider the output of a LayerNorm layer feeding into a linear projection. Channel j may have a maximum absolute value $\max |X_j| \approx 50$ while channel k has $\max |X_k| \approx 0.5$ — a $100\times$ difference. With a single per-tensor scale $s = 50/127 \approx 0.39$, channel k's values (≤ 0.5) are quantized to $\{-1, 0, 1\}$ — only 3 distinct levels out of 256, causing extreme precision loss.

6.3.2 Key Idea

SmoothQuant [2] migrates quantization difficulty from activations to weights by introducing a per-channel scale vector $\mathbf{s} \in \mathbb{R}^d$ that is absorbed into the adjacent (LayerNorm, Linear) pair without changing the mathematical output.

For a (LayerNorm $\rightarrow$ Linear) pair:

$$\mathbf{y} = \text{Linear}(\text{LN}(\mathbf{x})) = \mathbf{W} \cdot \text{LN}(\mathbf{x}) \tag{26}$$

Introduce scaling by $\mathbf{s}$ and its inverse:

$$\mathbf{y} = \underbrace{(\mathbf{W} \cdot \text{diag}(\mathbf{s}))}_{\mathbf{W}_{new}} \cdot \underbrace{\left(\text{diag}(\mathbf{s})^{-1} \cdot \text{LN}(\mathbf{x})\right)}_{\text{LN}_{new}(\mathbf{x})} \tag{27}$$

The scale $\mathbf{s}$ is absorbed into:

- **LayerNorm**: $\gamma_{new,j} = \gamma_j / s_j$ and $\beta_{new,j} = \beta_j / s_j$
- **Linear**: $W_{new,i,j} = W_{i,j} \cdot s_j$

This is a mathematically lossless transformation — the FP32 output is identical before and after applying the scales.

6.3.3 Scale Computation

The scale s_j for input channel j is computed from calibration statistics using the formula from the SmoothQuant paper (Eq. 1):

$$s_j = \frac{\max |X_j|^\alpha}{\max |W_j|^{1-\alpha}} \tag{28}$$

where:

- $\max|X_j| = \max_{\text{calib}}|X_{:,j}|$ is the per-channel maximum absolute activation, collected over the calibration set
- $\max|W_j| = \max_i|W_{i,j}|$ is the per-channel maximum absolute weight column norm
- $\alpha \in [0, 1]$ is a migration strength hyperparameter

When $\alpha = 0$: $s_j = 1/\max|W_j|$, pushing all difficulty to activations. When $\alpha = 1$: $s_j = \max|X_j|$, pushing all difficulty to weights. When $\alpha = 0.5$: s_j is the geometric mean, balancing difficulty equally between activations and weights.

We use $\alpha = 0.5$ (the default recommended value).

6.3.4 Activation Calibration

Per-channel maximum absolute activations are collected using PyTorch forward hooks. For each `nn.Linear` layer, a hook captures the input tensor and accumulates the running per-channel maximum across all calibration batches:

$$\hat{A}_j^{(k)} = \max\Big(\hat{A}_j^{(k-1)},\ \max_{\text{batch}}|X_{:,j}|\Big) \tag{29}$$

This is run for 64 calibration batches of 32 samples (2,048 samples total) from the validation set with no shuffling, to ensure consistent statistics.

6.3.5 Applied Layers

SmoothQuant is applied to the two most sensitive (LayerNorm, Linear) pairs in each decoder block:

1. $(\text{LN}_1, \text{attn.qkv})$ — the QKV projection receives LayerNorm-ed features before attention
2. $(\text{LN}_2, \text{ff.net}[0])$ — the FFN up-projection receives LayerNorm-ed features before the feed-forward

The attention output projection `attn.out_proj` and FFN down projection `ff.net[3]` are not SmoothQuant-ed because their inputs (attention-weighted values and post-GELU activations) do not directly follow a LayerNorm, so there is no adjacent norm layer to absorb the scale into.

6.4 AWQ: Activation-Aware Weight Quantization

6.4.1 Motivation

SmoothQuant uses a fixed formula (Eq. 28) to compute per-channel scales. AWQ [3] takes a more principled approach: it *searches* for the per-channel scale that directly minimizes the simulated INT8 quantization error for each weight matrix.

The key insight of AWQ is that not all weight channels are equally important. Channels j whose input activations $X_{:,j}$ tend to be large (salient channels) have an outsized influence on the output. If the weight column $W_{:,j}$ is poorly quantized, the output error is amplified by the large activation magnitude. AWQ therefore scales salient channels to reduce their quantization error at the expense of non-salient channels.

6.4.2 Pseudo-Quantization

The simulated per-output-channel symmetric INT8 quantization of a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ is:

$$Q(\mathbf{W})_{i,j} = \text{clamp}\left(\left\lfloor \frac{W_{i,j}}{s_i} \right\rceil, -128, 127 \right) \cdot s_i \qquad s_i = \frac{\max_j |W_{i,j}|}{127} \tag{30}$$

This is called *pseudo-quantization* because it simulates the quantization error in FP32 without actually converting to INT8.

6.4.3 Grid Search Objective

For a given per-input-channel scale vector $\mathbf{s} \in \mathbb{R}^{d_{in}}$ (normalized so that $\text{mean}(\mathbf{s}) = 1$), the AWQ-transformed weight matrix is:

$$\mathbf{W}_{scaled} = \mathbf{W} \cdot \text{diag}(\mathbf{s})^{-1} \qquad [W_{scaled}]_{i,j} = W_{i,j}/s_j \tag{31}$$

AWQ finds $\mathbf{s}$ that minimizes the round-trip quantization error:

$$\min_{\mathbf{s}} \|\mathbf{W} - Q(\mathbf{W}_{scaled}) \cdot \text{diag}(\mathbf{s})\|_F^2 \tag{32}$$

The scale is parameterized as $s_j = \hat{a}_j^{\alpha}$ where $\hat{a}_j = \max |X_{:,j}|$ is the activation importance and $\alpha \in [0, 1]$ is a scalar. The search space is:

$$\mathbf{s}(\alpha) = \frac{\hat{\mathbf{a}}^{\alpha}}{\text{mean}(\hat{\mathbf{a}}^{\alpha})} \tag{33}$$

The normalization by the mean ensures the overall weight magnitude is preserved. The grid search evaluates $n_{grid} + 1 = 21$ candidate values $\alpha \in \{0, 0.05, 0.10, \ldots, 1.0\}$ and selects the one minimizing Eq. 32.

6.4.4 Scale Application

Once the optimal $\mathbf{s}^*$ is found for each layer, it is folded into the adjacent (LayerNorm, Linear) pair identically to SmoothQuant, but with the inverse direction:

- **Linear**: $W_{new,i,j} = W_{i,j}/s_j^*$ (weight columns divided — reduces column norms)
- **LayerNorm**: $\gamma_{new,j} = \gamma_j \cdot s_j^*$ and $\beta_{new,j} = \beta_j \cdot s_j^*$ (absorbs the inverse scale)

Equivalence proof. Let $\mathbf{y} = \mathbf{W} \cdot \text{LN}(\mathbf{x})$. After folding $\mathbf{s}^*$:

$$\mathbf{y}_{new} = \frac{\mathbf{W}}{\text{diag}(\mathbf{s}^*)} \cdot (\text{diag}(\mathbf{s}^*) \odot \text{LN}(\mathbf{x})) = \mathbf{W} \cdot \text{LN}(\mathbf{x}) = \mathbf{y} \quad \checkmark \tag{34}$$

where $\odot$ denotes element-wise multiplication. The transformation is mathematically lossless; the FP32 accuracy of the transformed model is verified to be within 0.5% of the original before proceeding to INT8 quantization.

6.4.5 Relationship Between AWQ and SmoothQuant

Both methods fold per-channel scales into (LayerNorm, Linear) pairs. The difference lies in how the scale is computed:

Table 7: Comparison of SmoothQuant and AWQ scale computation.

	SmoothQuant	AWQ
Scale formula	$s_j = (\max \lvert X_j \rvert)^{\alpha} / (\max \lvert W_j \rvert)^{1-\alpha}$	$s_j = \hat{a}_j^{\alpha^*} / \text{mean}(\hat{\mathbf{a}}^{\alpha^*})$
How α is set	Fixed hyperparameter (0.5)	Grid-searched per layer
Objective	Balance activation/weight ranges	Minimize simulated INT8 error
Direction of scaling	Multiply weights, divide activations	Divide weights, multiply activations

In the combined pipeline (`quantize.py`), SmoothQuant is applied first to equalize activation/weight ranges, and then AWQ is applied on top of the SmoothQuant-transformed model to further minimize quantization error.

6.5 ONNX Export

After the weight transformations are applied, the model is exported to the ONNX format using `torch.onnx.export`:

- **Opset version**: 17 (supports all required operators)
- **Dynamic batch axis**: the batch dimension is marked dynamic so the model accepts any batch size at runtime
- **Constant folding**: enabled — static sub-expressions (positional embeddings, causal mask) are precomputed
- **Input name**: `"mel"`, shape `[batch, 101, 80]`
- **Output name**: `"logits"`, shape `[batch, 35]`

The exported graph is validated with `onnx.checker.check_model` to ensure structural correctness.

6.6 ONNX Runtime Static INT8 Quantization

6.6.1 Overview

After ONNX export, ONNX Runtime's `quantize_static` function is used to quantize the model. This inserts **QuantizeLinear (Q)** and **DequantizeLinear (DQ)** operators around each quantizable op, resulting in a *QDQ model* that can be executed with INT8 arithmetic on the CPU execution provider.

6.6.2 Calibration

Static quantization requires a calibration pass to determine the activation scales. A `CalibrationDataReader` feeds 64 batches of 32 validation samples (2,048 clips) through

the FP32 ONNX model. For each quantized tensor, the calibrator collects per-batch statistics and computes the quantization parameters.

The default calibration method is min-max: for tensor $\mathbf{t}$,

$$s = \frac{\max(\mathbf{t}) - \min(\mathbf{t})}{2 \cdot q_{max}} \qquad z = 0 \quad \text{(symmetric)} \tag{35}$$

6.6.3 Quantized Operator Selection

A critical implementation detail is restricting quantization to only `MatMul` and `Gemm` operators:

```
quantize_static(
    ...
    op_types_to_quantize=["MatMul", "Gemm"],
    ...
)
```

Listing 2: Restricting quantization to linear ops.

Without this restriction, the calibrator attempts to quantize the tensor produced by adding the causal mask (which in earlier versions contained $-\infty$ values), causing the computed scale to be $+\infty$ and all dequantized values to collapse to zero. By quantizing only `MatMul`/`Gemm`, the attention logits (which are outputs of `Add` operations with the causal mask) remain in FP32, and only the computationally expensive linear projections are quantized.

6.6.4 Quantization Settings

Table 8: ONNX Runtime static quantization settings.

Setting	Value
Format	QDQ
Activation type	QInt8 (signed)
Weight type	QInt8 (signed)
Per-channel weights	Yes
Activation symmetric	Yes
Weight symmetric	Yes
reduce_range	True
Op types quantized	MatMul, Gemm

Why symmetric? The inputs to the linear layers are LayerNorm outputs, which are approximately zero-mean (by construction of LayerNorm). Symmetric quantization with zero zero-point is appropriate for approximately symmetric distributions and avoids the overhead of computing zero-point offsets.

Per-channel weights. Each output channel i of each weight matrix $\mathbf{W}$ gets its own scale s_i, as opposed to a single scale for the whole tensor (per-tensor). Per-channel quantization reduces the maximum quantization error per channel and improves accuracy.

6.6.5 QDQ Graph Structure

For a `MatMul` node with inputs $\mathbf{X}$ (activation) and $\mathbf{W}$ (weight), the QDQ format inserts:

$$\text{MatMul}(\mathbf{X}, \mathbf{W}) \longrightarrow \text{DQ}(Q(\mathbf{X}), s_X) \cdot \text{DQ}(Q(\mathbf{W}), \mathbf{s}_W) \tag{36}$$

At runtime, ONNX Runtime's CPU execution provider fuses the Q/DQ pairs and executes the operation directly in INT8.

7 Deployment on Raspberry Pi 3

7.1 Hardware Constraints

Table 9: Raspberry Pi 3 Model B hardware.

Component	Specification
CPU	Broadcom BCM2837, 4× Cortex-A53 @ 1.2 GHz
RAM	1 GB LPDDR2
SIMD	ARM NEON (128-bit, 16× INT8 per cycle)
OS	Raspberry Pi OS 64-bit (arm64)

7.2 Runtime Configuration

The ONNX Runtime session is configured to maximize single-inference throughput:

- `intra_op_num_threads = 4`: all 4 Cortex-A53 cores are used within a single operator (e.g., matrix multiply is split across 4 threads)
- `inter_op_num_threads = 1`: only one operator runs at a time (ORT_SEQUENTIAL mode)
- `ORT_ENABLE_ALL`: all graph optimizations enabled (constant folding, node fusion, etc.)
- `enable_mem_pattern = True`: reuses memory allocations across inferences

7.3 Real-Time Inference

Real-time keyword spotting operates in **non-overlapping 1-second windows**:

1. Record 16,000 samples from the microphone
2. Compute the log-mel spectrogram (NumPy / librosa)
3. Run ONNX Runtime inference
4. Apply softmax and threshold at 70% confidence
5. Print the detected keyword (or "silence") with a confidence bar

The full pipeline (preprocessing + inference) runs in under 200 ms on the Pi 3, well within the 1-second window budget.

7.4 Preprocessing on the Pi

The inference scripts replicate the training preprocessing without PyTorch. If `librosa` is available, `librosa.feature.melspectrogram` is used. Otherwise a pure NumPy/SciPy fallback is used:

1. **STFT**: computed with a Hann window using `scipy.signal.get_window` and `numpy.fft.rfft`
2. **Mel filterbank**: triangular filters computed analytically from the mel-to-Hz conversion formula
3. **Log + normalization**: identical to training

The preprocessing parameters are saved alongside the ONNX model in `preprocess_config.json` so that the Pi can reconstruct the exact same spectrogram as used during training.

8 Summary

Table 10 summarizes the complete pipeline from raw audio to INT8 inference.

Table 10: End-to-end pipeline summary.

Stage	Tool	Key details
Preprocessing	torchaudio / librosa	16 kHz, 80-mel, 25 ms window, 10 ms hop, per-sample norm
Augmentation	torchaudio SpecAugment	FreqMask(10), TimeMask(20), training only
Architecture	PyTorch	Decoder-only transformer, 4 blocks, CLS at end, 2.1M params
Training	PyTorch	AdamW, OneCycleLR, label smoothing 0.1, 40 epochs
SmoothQuant	Custom PyTorch	$\alpha = 0.5$, folds into (LN, Linear) pairs, lossless
AWQ	Custom PyTorch	Grid search $\alpha \in [0, 1]$ (21 steps), folds into (LN, Linear)
ONNX export	torch.onnx	Opset 17, dynamic batch, constant folding
INT8 quant	ONNX Runtime	QDQ, per-channel, symmetric, MatMul/Gemm only
Inference	ONNX Runtime	4 threads, all optimizations, CPUExecutionProvider

References

[1] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. *Proc. Interspeech*, 2019.

[2] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *Proc. ICML*, 2023.

[3] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *Proc. MLSys*, 2024.

[4] P. Warden. Speech commands: A dataset for limited-vocabulary speech recognition. *arXiv:1804.03209*, 2018.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Proc. NeurIPS*, 2017.